

Hola Brian, David y Federico. Estuve revisando con detenimiento el repositorio completo, incluyendo la propuesta original, la versión unificada, el relevamiento y la propuesta de tutoría.

[GitHub - FedeiaTech/TFI-UTN--TechFlow--Colque-Gutierrez-Iacono · GitHub](#)

En términos generales considero que **TechFlow tiene una problemática válida y buenas condiciones para avanzar como Proyecto Final.**

Me parece especialmente positivo que no hayan cerrado la solución únicamente desde una idea técnica, sino que hayan realizado un relevamiento con negocios reales. También veo una evolución favorable desde la propuesta original, que estaba mucho más orientada a construir un ERP comercial, hacia un núcleo más específico relacionado con la **trazabilidad del ciclo de reparación.**

Para mí, ése es el punto que debemos preservar.

El proyecto debería poder responder muy bien:

¿qué equipo ingresó, qué problema tiene, quién lo está atendiendo, en qué etapa se encuentra, qué se diagnosticó, qué se presupuestó y cómo terminó esa reparación?

Antes de ampliar hacia compras, proveedores, analítica o IA quiero que ese circuito sea sólido.

Hay algunos puntos que necesitamos revisar.

1. **Relevamiento y evidencia.** Las tres experiencias obtenidas son un muy buen comienzo, pero todavía representan tres casos observados y no necesariamente “tres perfiles recurrentes del sector”. Además aparecen valores como 1,5–2 horas diarias de consultas, 40–60 minutos buscando repuestos o impactos de 20–35 % sobre el margen. Si esos números fueron proporcionados por los entrevistados, documenten su origen; si provienen de fuentes externas, incluyan las referencias; y si son hipótesis o estimaciones generadas durante el análisis, preséntenlas como tales. No necesitamos números llamativos para justificar el proyecto. Prefiero evidencia sencilla y comprobable. También eliminaría conclusiones del tipo “veredicto según IA”: la IA puede ayudarnos a analizar, pero las conclusiones profesionales tienen que ser asumidas y justificadas por ustedes.
2. **Recorten P0.** Actualmente el alcance incluye órdenes, clientes, presupuestos, pagos parciales, cuenta corriente, stock, proveedores, compras, roles, métricas e IA. Eso vuelve a acercar TechFlow a un ERP completo. Para el núcleo obligatorio concentraría el proyecto en algo semejante a: **cliente → equipo → orden → diagnóstico → presupuesto → aprobación/rechazo → reparación → listo → entrega.** Repuestos, cobros y roles pueden incorporarse sólo hasta el nivel necesario para completar ese flujo. Compras, analítica avanzada e IA pueden esperar.

3. **Definan las reglas de la orden de trabajo antes del modelo de datos.** Por ejemplo: ¿qué ocurre cuando un presupuesto es rechazado?, ¿puede modificarse después de enviado?, ¿se conserva el presupuesto anterior?, ¿qué ocurre si durante la reparación aparece una nueva falla?, ¿existe un estado “esperando repuesto”?, ¿cómo se trata un equipo que está listo pero permanece 30 días sin ser retirado? Este último punto me parece importante porque los equipos abandonados aparecieron precisamente en el relevamiento.
4. **No mezclen estado de reparación con estado financiero.** Entregado/Cobrado no debería necesariamente ser un único estado. Además actualmente dicen que manejarán cuenta corriente, pero al mismo tiempo que una orden no puede entregarse con saldo. Necesitamos consultar qué ocurre realmente en los talleres: si alguna vez entregan con deuda, la cuenta corriente tiene sentido; si nunca lo hacen, para PO probablemente alcance con total, pagos registrados y saldo pendiente.
5. **Revisen cuándo impacta un repuesto en stock.** Hoy plantean descontarlo al asociarlo a una orden. Pero si se reserva una pantalla y después el cliente rechaza el presupuesto, ¿realmente salió del stock? Quizás deban distinguir reservado/utilizado, o simplificar PO y descontar únicamente cuando el técnico confirma su utilización. La regla debe surgir del proceso real.
6. **No llamen “rentabilidad neta real” a algo que no lo es.** Si solamente calculan cobrado menos repuestos y mano de obra, pueden hablar de margen o resultado directo por reparación. Para hablar de ganancia neta faltarían alquileres, impuestos, servicios y muchos otros costos. También deberán definir cómo calculan el costo de mano de obra.
7. **Conecten el producto con el problema de WhatsApp.** Si uno de los dolores principales es responder constantemente “¿ya está?”, una aplicación únicamente interna no elimina esa consulta. No necesitan integrar la API de WhatsApp para PO. Podrían analizar un enlace público seguro donde el cliente consulte el estado de su orden, o una acción que prepare un mensaje con el estado actual para enviarlo manualmente. Algo semejante podría hacerse con el presupuesto y su aceptación/rechazo. Eso conectaría directamente la funcionalidad con el problema relevado.
8. **Eviten almacenar contraseñas o PIN de dispositivos sin una necesidad muy clara.** En el relevamiento aparecen contraseñas entre la información que puede perderse. Ese dato es especialmente sensible. Para el MVP mi primera recomendación sería no almacenarlo. Si un proceso real demuestra que es imprescindible, habrá que analizar específicamente acceso, protección y eliminación de esa información.
9. **Simplifiquen la decisión de persistencia.** Después de conocer mejor el dominio vemos relaciones muy naturales entre clientes, equipos, órdenes, presupuestos, pagos, repuestos y usuarios, además de necesidades de integridad, transacciones y reportes. Por eso PostgreSQL me parece actualmente la alternativa más natural, salvo que encuentren una razón concreta para otra cosa. Supabase puede servir perfectamente como PostgreSQL administrado; eso no obliga a utilizar toda su plataforma. No veo todavía una necesidad que justifique MongoDB ni dos motores de persistencia.

10. **Cierren también el backend.** En la propuesta de tutoría indican que Spring Boot ya está encaminado, mientras que otros documentos todavía dicen “Spring Boot o FastAPI”. Definan cuál utilizarán y documenten el nivel real de conocimiento de cada integrante en Java/Spring, Python/FastAPI, React/TypeScript, PostgreSQL y deployment. El objetivo no es utilizar la mayor cantidad de tecnologías, sino un stack que puedan defender y completar.
11. **Mantengan IA fuera del núcleo.** Me parece correcta la decisión de incorporarla sólo después de tener datos y un sistema funcionando. Tampoco considero que RAG sea actualmente una necesidad del proyecto. Si los datos son estructurados, el backend puede calcular determinísticamente métricas y un LLM eventualmente convertirlas en un resumen textual. RAG tendría sentido más adelante si existiera un corpus documental o conocimiento no estructurado que realmente necesitaríamos recuperar.

Para la próxima instancia prefiero que no sigan agregando módulos. Me interesa que elijan uno de los talleres relevados y documenten un caso completo de reparación: desde que el equipo entra hasta que se entrega, incluyendo diagnóstico, presupuesto, decisión del cliente, posibles cambios, repuestos utilizados y cobro.

A partir de ese caso quiero que definamos las reglas que nunca deberían poder violarse.

Después implementaría una primera porción vertical mínima:

crear cliente/equipo → abrir orden → diagnosticar → generar presupuesto → registrar aprobación → cambiar estado → marcar listo → cerrar la orden.

Si ese recorrido funciona correctamente, tendremos validado el corazón del proyecto y podremos decidir con mucha más seguridad qué módulos adicionales realmente aportan valor.

Considero que TechFlow está **bien encaminado y puede avanzar**. El principal desafío ahora no es incorporar más funcionalidades ni más tecnología, sino evitar que una problemática bien encontrada termine transformándose nuevamente en un ERP demasiado grande.

Saludos,
Oscar